FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Trabajo Práctico 1
File Transfer

[TA048] - Redes
Cátedra Hamelin y Balbiano
Primer cuatrimestre de 2026

Nombre	Padrón	Correo Electrónico
Agustín Abitabile	111081	aabitabile@fi.uba.ar
Chiara López Angelini	111225	clopezan@fi.uba.ar
Felipe Quero Retamal	111374	fquero@fi.uba.ar
Lucas Dondo	112527	ldondo@fi.uba.ar
Matías Llorens	112469	mllorens@fi.uba.ar

Índice

1. Introducción **2**

2. Hipótesis y suposiciones realizadas **3**
2.1. Serialización del segmento TCP con `pickle` . . . 3
2.2. Tamaño del Segmento . . . 3
2.3. Timeout . . . 3
2.4. Finalización de Conexión (Teardown) . . . 3
2.5. *Checksum* . . . 3

3. Implementación **4**
3.1. Gestión de Errores . . . 4
3.2. Automatización y Usabilidad . . . 4
3.3. Arquitectura Multihilo del Servidor . . . 5

4. Pruebas **6**
4.1. Escenario de Red (Topología) . . . 6
4.2. Metodología de Prueba . . . 6
4.3. Verificación de Integridad . . . 6
4.4. Prueba de Ejecución . . . 7

5. Preguntas a responder **8**
5.1. Describa la arquitectura cliente-servidor . . . 8
5.2. ¿Cuál es la función de un protocolo de capa de aplicación? . . . 8
5.3. Detalle el protocolo de aplicación desarrollado en este trabajo . . . 9
5.4. TCP y UDP: servicios, características y casos de uso . . . 10
5.5. Mecanismos de control de congestión del protocolo desarrollado . . . 11
5.6. ¿Cuál protocolo desarrollado envía archivos en menor tiempo? . . . 11

6. Dificultades encontradas **13**

7. Conclusión **14**

1. Introducción

El presente trabajo práctico tiene como objetivo el diseño e implementación de una aplicación de red con arquitectura cliente-servidor que permita la transferencia de archivos entre hosts. Concretamente, se implementaron dos operaciones: **UPLOAD**, que transfiere un archivo desde el cliente hacia el servidor, y **DOWNLOAD**, que transfiere un archivo desde el servidor hacia el cliente.

Dado que la comunicación entre procesos se realiza sobre UDP como protocolo de capa de transporte, fue necesario implementar mecanismos de transferencia confiable (*Reliable Data Transfer*, RDT) por encima de este protocolo. Para ello se desarrollaron dos variantes: una basada en **Stop & Wait** y otra basada en **Go-Back-N**, ambas capaces de garantizar la entrega de paquetes ante pérdidas de hasta el 10 % en los enlaces.

Las aplicaciones fueron desplegadas y probadas en una topología de red simulada con Mininet, lo que permitió reproducir distintas condiciones de red de forma controlada. Además, se diseñó un protocolo de capa de aplicación propio que especifica los mensajes intercambiados entre cliente y servidor para cada operación.

El presente informe describe las decisiones de diseño e implementación tomadas, los resultados obtenidos y las conclusiones derivadas de la comparación entre ambos protocolos de recuperación de errores.

2. Hipótesis y suposiciones realizadas

2.1. Serialización del segmento TCP con `pickle`

Una de las suposiciones iniciales fue que podríamos implementar el segmento TCP como una clase de Python y luego enviarla como *bytes* serializados usando la librería `pickle`. Lo bueno de esta implementación, es que, en lugar de hacerlo todo con bits editados de manera directa, nos permite un control más elegante y con código más *pythonic*, de no tan bajo nivel. Lo cierto es que esta implementación termina usando más *bytes* que los 40 usados en TCP, y esto hace que nuestro MSS utilizado sea menor, lo cual requiere enviar más paquetes para transmitir la misma información que en el caso del TCP estándar.

2.2. Tamaño del Segmento

Se fijó un *Maximum Segment Size* (MSS) de 1350 bytes para la carga útil (*payload*). Esta decisión parte de la hipótesis de que el encabezado generado por la serialización con `pickle`, sumado a los encabezados UDP (8 bytes) e IP (20 bytes), no superará el *Maximum Transmission Unit* (MTU) estándar de 1500 bytes. El objetivo primordial de esta restricción es evitar la fragmentación en capas inferiores de la red, lo cual degradaría significativamente el rendimiento del protocolo RDT debido a la sobrecarga de reensamblado.

2.3. Timeout

Tras realizar pruebas empíricas en el entorno de Mininet, se determinó que un valor de `TIMEOUT` fijo de 0.05 segundos es suficiente para cubrir el *Round Trip Time* (RTT) promedio de la red virtual, incluso bajo condiciones de pérdida del 10 %. Se asume que este valor representa un balance óptimo: es lo suficientemente alto para evitar retransmisiones espurias (falsos positivos) y lo suficientemente bajo para no penalizar el rendimiento global tras una pérdida real de paquetes.

Vale mencionar que se ha hecho una implementación de *timeout* dinámico en base a la sección 3.5.3 del libro de Kurose. Esto fue con el objetivo de mejorar la *performance*, pero no logramos dar con los valores iniciales y/o actualizaciones adecuadas para que éste algoritmo efectivamente aumente la velocidad del sistema. Por ende, terminamos descartando la implementación en pos del *timeout* estático ya mencionado.

2.4. Finalización de Conexión (Teardown)

Dada la naturaleza no fiable del canal, se asume que la red puede perder el último ACK de la fase de cierre (el problema de los dos generales). Por lo tanto, el protocolo supone que:

- Tras alcanzar un número máximo de reintentos (`MAX_RETRIES`) sin recibir respuesta durante la fase de `FIN`, ambos extremos pueden dar la conexión por cerrada de manera segura.
- Esta liberación de recursos es válida bajo la suposición de que el extremo opuesto eventualmente llegará a la misma conclusión al agotar sus propios temporizadores, garantizando que el sistema no quede en un estado de bloqueo indefinido.

2.5. *Checksum*

En un momento dado, se supuso la necesidad de agregar el *checksum* a nuestra clase de Python `Segment`, pues estamos buscando alcanzar un ***reliable*** *data transfer* y por consecuencia un envío . Sin embargo, prontamente recordamos que el segmento UDP (mediante el cual mandamos nuestro `Segment` en su *payload*) ya cuenta con un *checksum* y su verificación.

Por ende, nos quedamos tranquilos de que el tema de corrupción de segmentos es algo que ya quedó delegado en la capa UDP sobre la cual operamos.

3. Implementación

La implementación de la solución se basó en una arquitectura modular, separando la lógica de los protocolos de transporte (*Stop & Wait* y *Go-Back-N*) de la lógica de aplicación y la gestión de archivos.

3.1. Gestión de Errores

Un pilar fundamental del desarrollo fue garantizar que la aplicación sea robusta frente a fallos de red, errores del usuario o estados inconsistentes del sistema de archivos.

Validación Preventiva (Módulo Parser)

Para evitar errores en tiempo de ejecución, se implementó una validación rigurosa de los argumentos de entrada mediante el módulo `argparse` en `parser.py`:

- **Restricción de Protocolo:** Se utilizó el parámetro `choices` para asegurar que el usuario solo pueda seleccionar protocolos implementados (`stop-and-wait` o `go-back-n`).
- **Integridad de Parámetros:** Se marcaron como `required` argumentos críticos, como la ruta de origen (`-src`) en los procesos de *upload*, impidiendo que el protocolo inicie sin los datos necesarios.
- **Gestión de Verbosity:** Se implementó un grupo mutuamente excluyente para los flags `-q` (quiet) y `-v` (verbose), permitiendo el control de los logs sin conflictos de configuración.

Tratamiento de Archivos Corruptos e Incompletos en Download

En operaciones de red, es común que una transferencia se interrumpa. Para evitar dejar "basura" en el sistema del cliente, el archivo `download.py` implementa una política de limpieza automática:

- **Eliminación Post-Fallo:** Si ocurre cualquier excepción durante el método `client_recv` (un error de conexión o un timeout persistente), se ejecuta un bloque `except` que elimina mediante `os.remove` el archivo que se estaba escribiendo. Esto garantiza que el usuario no intente abrir un archivo incompleto o corrupto.
- **Limpieza de Directorios:** Adicionalmente, el sistema intenta ejecutar `os.rmdir` sobre el directorio de destino. Si la carpeta quedó vacía tras borrar el archivo fallido, se elimina para mantener la higiene del sistema de archivos local; de lo contrario, la excepción `OSError` es silenciada para no interrumpir el flujo.

Gestión de Archivos en Upload

En el lado del emisor, en el archivo `upload.py`, se añadieron manejadores específicos para errores comunes de archivos, informando al usuario de manera descriptiva:

- **FileNotFoundError:** Se captura para informar que la ruta de origen es inválida.
- **PermissionError:** Se gestiona mediante una constante `PERMISSION_DENIED` para casos donde el usuario no tiene privilegios de lectura.

3.2. Automatización y Usabilidad

La aplicación fue diseñada para requerir la mínima intervención manual en la preparación del entorno de red:

- **Creación Dinámica de Directorios:** Tanto el servidor (`server.py`) como el cliente de descarga utilizan `os.makedirs(..., exist_ok=True)`. Esto permite que, si el usuario especifica una carpeta de almacenamiento o descarga que no existe, el programa la cree automáticamente antes de iniciar la transferencia.

3.3. Arquitectura Multihilo del Servidor

El servidor implementa un modelo concurrente para atender múltiples clientes simultáneamente. Al recibir un paquete `SYN`, el proceso principal instancia un objeto de la clase `Server`, el cual delega cada conexión a un hilo independiente mediante `threading.Thread`.

- **Aislamiento:** Cada hilo gestiona su propio socket de conexión (`connection_socket`) y su contexto de protocolo, asegurando que un error en una transferencia no afecte a las demás.
- **Control de Salida:** El uso de `KeyboardInterrupt` permite un cierre controlado del servidor, asegurando que el `welcoming_socket` se libere correctamente para futuras ejecuciones.

4. Pruebas

Para validar el correcto funcionamiento de los protocolos *Stop & Wait* y *Go-Back-N*, se diseñó un escenario de pruebas controlado utilizando **Mininet**. El objetivo fue observar el comportamiento del sistema bajo condiciones de red adversas y carga simultánea de múltiples clientes.

4.1. Escenario de Red (Topología)

Se implementó una topología mediante un script de Python (`topology.py`) que utiliza un switch virtual (`OVSController`) para conectar 5 nodos: un servidor central (`server`) y cuatro clientes (`client1` a `client4`).

Configuración de Enlaces: Se aplicó una pérdida de paquetes del 10 % específicamente en el enlace del servidor para simular una red inestable:

```
self.addLink(server, switch, cls=TCLink, loss=10)
```

Esta configuración es estratégica: al afectar el enlace troncal del servidor, tanto los paquetes de datos (*Data*) como las confirmaciones (*ACKs*) de todos los clientes sufren pérdidas. Esto fuerza la activación constante de los mecanismos de retransmisión.

4.2. Metodología de Prueba

Se generaron archivos de prueba de 5MB con datos aleatorios utilizando el comando `dd` para asegurar que el contenido no fuera predecible y facilitar la detección de errores de corrupción de bits. Ejemplo:

```
md5sum OG.bin storage/UploadedWithGBN.jpg
```

La ejecución consistió en los siguientes pasos:

1. **Levantamiento del Servidor:** Ejecutado en el host `server` escuchando en la IP `10.0.0.5` y puerto `12000`.
2. **Ejecución Simultánea:** Se lanzaron 4 operaciones en paralelo desde terminales `xterm` independientes para estresar el manejo de hilos del servidor:
   - **Cliente 1:** *Upload* mediante *Stop & Wait*.
   - **Cliente 2:** *Upload* mediante *Go-Back-N*.
   - **Cliente 3:** *Download* mediante *Stop & Wait*.
   - **Cliente 4:** *Download* mediante *Go-Back-N*.

4.3. Verificación de Integridad

Tras finalizar las transferencias, se procedió a comparar los archivos originales con los archivos transferidos. Se utilizó el algoritmo **MD5** para garantizar que los archivos fueran idénticos, asegurando que el 10 % de pérdida fue mitigado correctamente por el protocolo RDT. Ejemplo:

```
md5sum OG.bin storage/UploadedWithGBN.jpg
```

4.4. Prueba de Ejecución

Para validar la implementación, se realizó una prueba ejecutando cuatro transferencias simultáneas (dos cargas y dos descargas) utilizando ambos protocolos (*Stop & Wait* y *Go-Back-N*), de archivos de 5MB. Esta prueba permite verificar no solo la integridad de los datos bajo una pérdida del 10 %, sino también la capacidad del servidor para gestionar múltiples hilos de ejecución concurrentes.

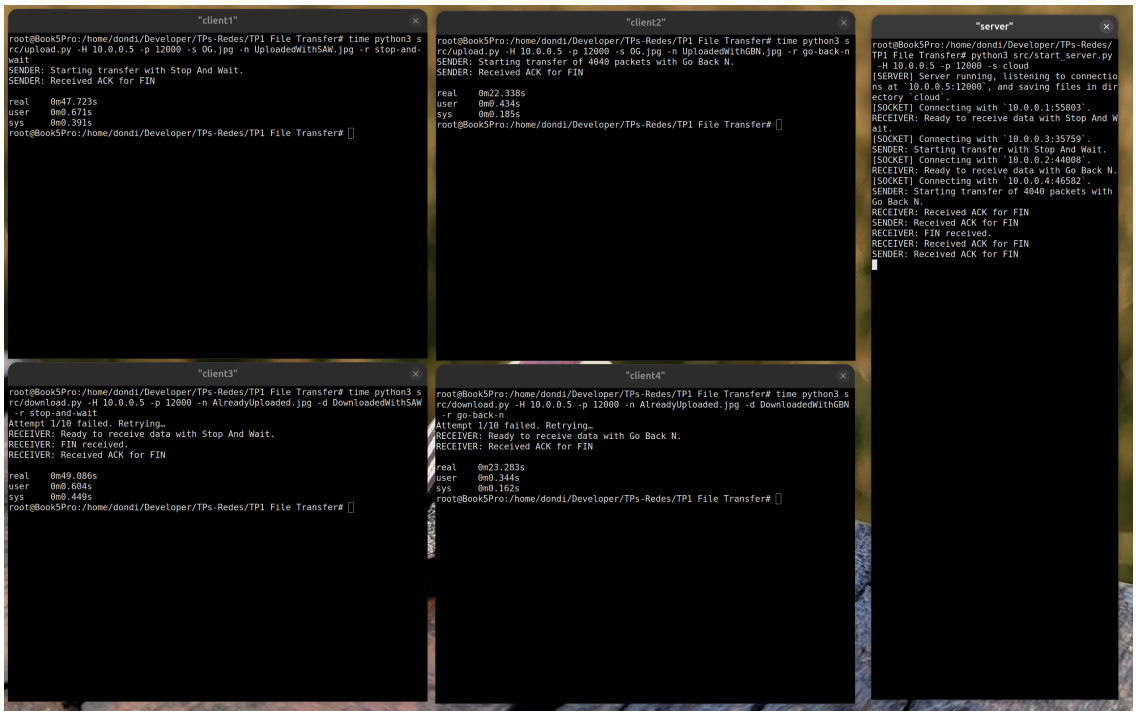


Figura 1: Ejecución simultánea de clientes y servidor en el entorno Mininet.

Como se aprecia en la Figura 1, la captura de pantalla del entorno de pruebas refleja los siguientes hitos:

- **Servidor Multihilo:** En la ventana central (`server`), se observan los logs de depuración indicando la creación de nuevos hilos para atender a cada cliente de forma independiente mediante sockets UDP conectados.
- **Progreso de Clientes:** Las terminales de los clientes (`client1` a `client4`) muestran barras de progreso activas, confirmando que las retransmisiones por *timeout* ocurren de manera transparente al usuario.
- **Medición de Tiempos:** Se utiliza el comando `time` para obtener una métrica precisa del tiempo de ejecución real, lo que permite realizar la comparativa de rendimiento entre los protocolos de ventana deslizante y los de parada y espera.
- **Finalización Limpia:** Se observa el mensaje `Teardown complete`, lo que garantiza que los estados de conexión se cerraron correctamente y los recursos del sistema fueron liberados a pesar de la inestabilidad de la red.

5. Preguntas a responder

5.1. Describa la arquitectura cliente-servidor

La arquitectura cliente-servidor es uno de los modelos principales de organización de aplicaciones de red. En este modelo se definen dos roles importantes para el funcionamiento de la aplicación: **Cliente** y **Servidor**.

Servidor

El servidor es la pieza central de la arquitectura; básicamente es el medio por el cual los clientes deben comunicarse y mandar mensajes.

El servidor debe estar siempre encendido para responder todas las solicitudes que recibe de los clientes, mientras la aplicación esté en funcionamiento. Para que siempre pueda recibir solicitudes, el servidor debe tener una dirección IP conocida y estática, ya que si cambia los clientes no podrán comunicarse.

Un servidor puede tener una base de datos donde estén almacenados los datos de cada cliente y otros datos necesarios. Pueden tener miles de clientes conectados al mismo tiempo, razón por la cual muchos son alojados en centros de datos (*data centers*).

Cliente

Un cliente es básicamente un dispositivo que tiene un usuario: una computadora, celular, etc. Son los que inician la comunicación con el servidor para realizar una petición, y el servidor les responde.

No tienen que estar siempre conectados con el servidor, ya que el servidor nunca le enviará un mensaje al cliente sin que este haya hecho una petición primero.

Una de las principales diferencias de este modelo con el P2P es que los clientes no pueden intercambiar mensajes entre sí; deben hacerlo por medio del servidor. Cada mensaje lo recibe el servidor y este lo reenviará al otro cliente. Claramente ambos deben estar conectados, o al menos el servidor debe tener la información del cliente al cual se desea enviar un mensaje.

5.2. ¿Cuál es la función de un protocolo de capa de aplicación?

La aplicación es básicamente el motivo por el cual el resto de las capas tienen existencia, para que exista una aplicación que el usuario pueda usar y disfrutar. Esta aplicación se comunica mediante procesos, donde se intercambian mensajes a través de los *sockets* como interfaz. Al haber un intercambio, debe existir un conjunto de reglas sobre cómo estos mensajes están estructurados, qué significado tiene cada campo y cuándo se envían. A partir de esto surgen los **protocolos de capa de aplicación**.

Un **protocolo** de capa de aplicación es el encargado de definir cómo los procesos de una aplicación —que pueden estar en distintos *hosts*— intercambian mensajes. Específicamente define:

- **Tipos de mensajes intercambiados:** por ejemplo, *request–response*.
- **Sintaxis de mensajes:** por ejemplo, los campos presentes en un mensaje.
- **Semántica de los campos:** el significado propio del mensaje.
- **Reglas de envío y recepción:** cuándo un proceso puede enviar o recibir mensajes.

Podemos relacionar la capa de aplicación con la consigna anterior, donde se habló sobre la arquitectura cliente-servidor. La web está basada en este modelo y permite a los usuarios llevar a cabo distintas tareas, como obtener documentos, ver videos, imágenes, entre otros.

La web está formada por varios componentes: navegadores, el estándar para el formato de documentos (HTML) y servidores. El protocolo de capa de aplicación en este caso es **HTTP**,

donde se especifica, por ejemplo, cómo los mensajes deben ser intercambiados entre un navegador y un servidor.

Otros protocolos de capa de aplicación conocidos son: *Simple Mail Transport Protocol* (SMTP), *File Transfer Protocol* (FTP) y *Domain Name System* (DNS).

5.3. Detalle el protocolo de aplicación desarrollado en este trabajo

El protocolo de aplicación desarrollado opera sobre UDP y está compuesto por tres fases bien definidas: **handshake**, **transferencia de datos** y **teardown**. Todos los mensajes intercambiados utilizan la clase `Segment`, serializada con `pickle`, como unidad de comunicación.

Estructura de los mensajes

Cada mensaje es una instancia de `Segment`, que contiene los siguientes campos:

- `seq_num`: número de secuencia del segmento.
- `ack_num`: número de confirmación.
- `flags`: diccionario con tres banderas booleanas: `SYN`, `ACK` y `FIN`.
- `payload`: datos del archivo en bytes, o metadatos serializados en el caso del handshake.

El tamaño máximo de transferencia por unidad es el MTU (*Maximum Transmission Unit*) de 1500 bytes, para evitar fragmentación en capas inferiores. El tamaño máximo del payload (`MAX_PAYLOAD_SIZE`) es de 1350 bytes, valor inferior al estándar de 1460 bytes, debido al overhead adicional introducido por la serialización con `pickle`.

Fase 1: Handshake

El establecimiento de conexión sigue un esquema de *three-way handshake* adaptado. A diferencia del TCP estándar, el primer mensaje ya transporta en su payload los metadatos de la sesión, implementados como una instancia de la clase `Metadata`, que contiene:

- La operación solicitada: `UPLOAD` o `DOWNLOAD`.
- El nombre del archivo involucrado.
- El protocolo de recuperación de errores elegido: `stop-and-wait` o `go-back-n`.

La secuencia del handshake es la siguiente:

1. El cliente envía un segmento con `SYN=1` y la `Metadata` serializada en el payload.
2. El servidor responde con `SYN=1, ACK=1, ack_num=1`.
3. El cliente confirma con `ACK=1, ack_num=1` y ejecuta `connect()` al servidor, estableciendo el socket dedicado para la sesión.

El servidor mantiene un *welcoming socket* en el puerto principal que escucha nuevas conexiones. Al recibir un `SYN`, crea un socket dedicado para ese cliente y lanza un hilo independiente para atenderlo, lo que permite atender múltiples clientes de forma concurrente. Si el `SYN` proviene de un cliente ya en curso (por retransmisión), el servidor simplemente reenvía el `SYN+ACK` sin crear un nuevo hilo.

El cliente reintenta el handshake hasta `MAX_TIMEOUT_HANDSHAKE = 10` veces antes de lanzar un error de conexión.

Fase 2: Transferencia de datos

Una vez establecida la conexión, la transferencia se realiza mediante uno de los dos protocolos de recuperación de errores:

- **Stop & Wait:** el emisor envía un segmento y espera el `ACK` correspondiente antes de enviar el siguiente. Utiliza números de secuencia alternantes (0 y 1). Ante un timeout de 50 ms sin recibir `ACK`, retransmite el segmento.
- **Go-Back-N:** el emisor mantiene una ventana deslizante de tamaño fijo de 10 paquetes. Envía hasta 10 segmentos sin esperar confirmación individual. Si ocurre un timeout, retransmite desde el paquete base de la ventana. El receptor descarta paquetes fuera de orden y reenvía el último `ACK` válido.

En ambos casos, el timeout es de **50 ms** (`TIMEOUT = 0.05`), valor determinado empíricamente para obtener el mejor balance entre rendimiento y tolerancia a pérdidas.

Fase 3: Teardown

El cierre de la conexión es simétrico y está implementado en los métodos `_teardown_sender` y `_teardown_recv`. La secuencia es:

1. El emisor envía `FIN` con su número de secuencia final y espera un `ACK`.
2. El receptor, al detectar el `FIN`, envía su propio `ACK` y luego envía un `FIN` propio.
3. El emisor confirma el `FIN` del receptor con un `ACK` final.

Para manejar el caso en que el último `ACK` se pierda, el emisor permanece en escucha y reenvía el `ACK` si recibe un `FIN` duplicado. Además, se contempla explícitamente el caso de *deadlock* `FIN-FIN`: si el emisor recibe un `FIN` mientras espera el `ACK` de su propio `FIN`, envía un `ACK` y termina sin esperar más. El número máximo de reintentos en el teardown es `MAX_RETRIES = 8`.

5.4. TCP y UDP: servicios, características y casos de uso

La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP.

TCP

El protocolo TCP (*Transmission Control Protocol*) es un protocolo orientado a la conexión, diseñado para ofrecer la máxima fiabilidad posible. Ofrece los siguientes servicios y características:

- **Transferencia confiable:** garantiza que los mensajes enviados desde un host lleguen a su destino sin errores y en el orden correcto.
- **Control de flujo:** evita que el receptor se sature si es más lento receptando datos que el emisor enviándolos.
- **Control de congestión:** detecta si la red está colapsada y reduce la tasa de envío para no saturar más el tráfico.
- **Servicio *full-duplex*:** ambos hosts pueden enviar y recibir mensajes al mismo tiempo.
- **Establecimiento de conexión:** utiliza el *three-way handshake* antes de iniciar la transferencia de datos. El cliente envía una solicitud de conexión al servidor (`SYN`), el servidor confirma y envía su propia solicitud (`SYN+ACK`), y el cliente confirma (`ACK`).

Este protocolo es apropiado en servicios donde lo más importante es que el mensaje llegue sin errores ni datos extraviados, aunque la velocidad sea secundaria. Ejemplos: navegadores web (HTTP), correo electrónico (SMTP) y transferencia de archivos (FTP).

UDP

El protocolo UDP (*User Datagram Protocol*) es un protocolo sin conexión, extremadamente rápido y liviano. Ofrece los siguientes servicios y características:

- **Sin conexión:** no requiere establecer una conexión previa; simplemente se envían y reciben datos hacia una dirección en particular.
- **Baja latencia:** al no requerir confirmaciones (*ACKs*), es mucho más rápido que TCP.
- **Sin orden garantizado:** los paquetes pueden llegar en distinto orden.
- **No fiable:** no hay garantía de entrega ni de orden de llegada. Si un paquete no llega, es simplemente descartado.

Este protocolo se utiliza en servicios donde lo más importante es la rapidez y donde perder algunos paquetes no es crítico. Ejemplos: *streaming* de video, juegos en línea y videollamadas.

5.5. Mecanismos de control de congestión del protocolo desarrollado

El protocolo desarrollado **no implementa mecanismos de control de congestión**, por lo que no aportamos al bien general de la red, dado que es para eso que existe el control de congestión. Sin embargo, hasta la cantidad de *hosts* que probamos (5), ésto no impuso ninguna penalización al estar todos transmitiendo información, por lo cual no nos pareció algo de que nos debiéramos preocupar de momento.

Si fuéramos a implementar control de congestión, la manera más sencilla en nuestra situación, sería mediante *end-to-end congestion control*, no *network-assisted congestion control*, para no tener dependencias externas a nuestro sistema.

De hecho, como comentario extra, a último momento logramos implementar un *fast restransmit congestion control* que bajó el tiempo de transmición de archivos de 5MBs a 5 segundos; pero por falta de tiempo para *testear*, decidimos dejarlo afuera de esta entrega.

5.6. ¿Cuál protocolo desarrollado envía archivos en menor tiempo?

Como se puede apreciar en la imagen incluida en la sección de pruebas (Figura 1), los tiempos de ejecución obtenidos mediante el comando `time` demuestran una clara diferencia de rendimiento entre ambas implementaciones.

Los resultados obtenidos bajo una condición de red con 10 % de pérdida de paquetes, utilizando archivos de 5MB, se resumen en la siguiente tabla.

Aclaración: El throughput promedio de cada transferencia se calculó como throughput = (tamaño en bits) / (tiempo en segundos).

Protocolo	**Operación**	**Tiempo (s)**	**Throughput (Mbps)**
Stop & Wait	Upload	47.723	0.838
Stop & Wait	Download	49.086	0.815
Go-Back-N (W=10)	Upload	22.338	1.791
Go-Back-N (W=10)	Download	23.283	1.718

Cuadro 1: Tiempos de transferencia de archivos de 5MB en una red con 10 % de pérdida de paquetes.

Además, con el objetivo de establecer una base para la compración, se realizaron mediciones en un escenario de control sin degradación de red (0 % de pérdida), también con archivos de 5MB. Estos valores permiten observar el comportamiento de los protocolos en condiciones ideales, donde las retransmisiones son nulas.

Protocolo	**Operación**	**Tiempo (s)**	**Throughput (Mbps)**
Stop & Wait	Upload	0.573	69.81
Stop & Wait	Download	0.864	46.30
Go-Back-N (W=10)	Upload	0.343	116.76
Go-Back-N (W=10)	Download	0.390	102.56

Cuadro 2: Tiempos de transferencia de archivos de 5M en una red con 0 % de pérdida de paquetes.

Luego se realizaron pruebas con archivos de 10MB Y 15MB, bajo una condición de red con 10 % de pérdida de paquetes, para analizar los resultados:

Protocolo	**Operación**	**Tiempo (s)**	**Throughput (Mbps)**
Stop & Wait	Upload	92.145	0.868
Stop & Wait	Download	97.185	0.823
Go-Back-N (W=10)	Upload	46.741	1.711
Go-Back-N (W=10)	Download	48.439	1.651

Cuadro 3: Tiempos de transferencia de archivos de 10M en una red con 10 % de pérdida de paquetes.

Protocolo	**Operación**	**Tiempo (s)**	**Throughput (Mbps)**
Stop & Wait	Upload	147.114	0.816
Stop & Wait	Download	141.768	0.847
Go-Back-N (W=10)	Upload	68.280	1.757
Go-Back-N (W=10)	Download	70.257	1.707

Cuadro 4: Tiempos de transferencia de archivos de 15M en una red con 10 % de pérdida de paquetes.

En conclusión, **Go-Back-N es el protocolo que envía archivos en menor tiempo en todos los escenarios evaluados**, logrando aproximadamente el doble de throughput bajo condiciones de pérdida del 10 %.

6. Dificultades encontradas

Uno de los mayores problemas que tuvimos fue con la reducción del tiempo que tardaba el programa en realizar sus operaciones. Sin *paquet loss*, los tiempos rondaban 1 segundo. Pero al agregar el 10 % de pérdida de paquetes, terminábamos yéndonos lejos de los máximos 2 minutos requeridos por la cátedra para archivos de 5MBs.

Viendo esto, hicimos un análisis exhaustivo del programa. Nuestro primer candidato a ser el problema era `pickle`, pero luego de implementar un segmento sencillo basado en *bytes* e integrarlo rápidamente con el sistema y aumentar el `MAX_PAYLOAD_SIZE` de manera acorde, no se notó mejora alguna. Luego estuvimos verificando todo el proceso de envío y recepción de paquetes y ACKs a lo largo del protocolo, pero sin encontrar nada que llame la atención negativamente.

También probamos distintas configuraciones de `MTU` y `MAX_PAYLOAD_SIZE`, pero al reducir mucho el tamaño del *payload*, terminábamos desperdiciando *bytes* en *headers* extra, y si aumentábamos mucho el `MTU` ocurría que por fragmentación se agregaba un *delay* notable.

Luego nos dimos cuenta que el programa únicamente era lento cuando había pérdida de paquetes. Fue entonces que verificamos lo relacionado a la gestión de pérdida de paquetes. Fue así que terminamos eliminando el *timeout* dinámico en pos de un valor estático (ya mencionado) que nos otorgó el punto justo para obtener la mejor *performance* en el entorno que tenemos.

7. Conclusión

Para cerrar, se podría decir que fue un buen desarrollo para terminar de entender cómo se lleva a cabo cada parte del transporte; pero también para notar cómo hay pequeñas variaciones que pueden generar grandes cambios en el funcionamiento del protocolo (ya sea desde el lado de tiempos o de por ejemplo casos donde algún *host* se queda esperando paquetes que nunca llegarán). Con esto, hemos visto lo importante de que un protocolo sea claro, tenga las responsabilidades bien marcadas y ofrezca una simple interfaz para los *layers* más altos. Sin esto, se duplican acciones, no se gestionan otras, y se multiplican los errores en casos borde.